

L1 and L2 Normalization and Regularization

Dr. Adnan Arefeen

1 L1 and L2 Normalization

Normalization changes the numerical values of a vector by rescaling it. It is usually applied to input vectors, feature vectors, embeddings, or layer activations.

Let

$$\mathbf{x} = [3, 4].$$

1.1 L1 Normalization

The L1 norm of a vector is the sum of the absolute values of its components:

$$\|\mathbf{x}\|_1 = |x_1| + |x_2| + \cdots + |x_n|.$$

For

$$\mathbf{x} = [3, 4],$$

we get

$$\|\mathbf{x}\|_1 = |3| + |4| = 7.$$

Therefore the L1-normalized vector is

$$\mathbf{x}' = \frac{\mathbf{x}}{\|\mathbf{x}\|_1} = \left[\frac{3}{7}, \frac{4}{7} \right] = [0.4286, 0.5714].$$

Notice that

$$|0.4286| + |0.5714| = 1.$$

Meaning of L1 normalization. L1 normalization rescales a vector so that the sum of the absolute values of its entries becomes 1.

1.2 L2 Normalization

The L2 norm of a vector is its Euclidean length:

$$\|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}.$$

For

$$\mathbf{x} = [3, 4],$$

we get

$$\|\mathbf{x}\|_2 = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = 5.$$

Therefore the L2-normalized vector is

$$\mathbf{x}' = \frac{\mathbf{x}}{\|\mathbf{x}\|_2} = \left[\frac{3}{5}, \frac{4}{5} \right] = [0.6, 0.8].$$

Notice that

$$0.6^2 + 0.8^2 = 1.$$

Meaning of L2 normalization. L2 normalization rescales a vector so that its Euclidean length becomes 1.

1.3 Does Normalization Really Change the Input?

Yes. For example,

$$[3, 4] \rightarrow [0.6, 0.8]$$

under L2 normalization. The numerical values change. However, the direction of the vector remains the same because

$$[0.6, 0.8] = \frac{1}{5}[3, 4].$$

Thus, normalization removes magnitude information while preserving the direction or relative pattern of the vector.

Example. Consider two vectors:

$$\mathbf{x}_1 = [3, 4], \quad \mathbf{x}_2 = [300, 400].$$

Their L2 norms are

$$\|\mathbf{x}_1\|_2 = 5, \quad \|\mathbf{x}_2\|_2 = 500.$$

After L2 normalization:

$$\mathbf{x}'_1 = [0.6, 0.8], \quad \mathbf{x}'_2 = [0.6, 0.8].$$

Although the second vector has a much larger magnitude, both vectors have the same direction. L2 normalization makes them identical.

1.4 Normalization versus Regularization

Normalization and regularization are often confused because both use L1 and L2 mathematical forms. But they are not the same.

Concept	Applied to	Purpose
L1/L2 normalization	Input vectors, embeddings, activations	Rescale vectors
L1/L2 regularization	Model weights in the loss function	Reduce overfitting by controlling learned weights

2 Why Regularization is Needed

A model is trained to minimize prediction error. Without regularization, the model may learn overly large or unnecessary weights that fit the training data too closely. This is called overfitting.

Suppose we predict exam marks using

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3,$$

where

$$x_1 = \text{study hours}, \quad x_2 = \text{attendance}, \quad x_3 = \text{student ID}.$$

A good model should use study hours and attendance. It should not use student ID. However, with a small or noisy dataset, the model may accidentally learn

$$\hat{y} = 5x_1 + 2x_2 + 0.4x_3.$$

The term

$$0.4x_3$$

means student ID affects the prediction, which is undesirable. Regularization discourages this kind of unnecessary dependence.

2.1 Large Weights Cause Sensitivity

Consider

$$\hat{y} = 100x.$$

If

$$x = 1.00,$$

then

$$\hat{y} = 100.$$

If

$$x = 1.01,$$

then

$$\hat{y} = 101.$$

A tiny input change creates a large output change.

Now compare with

$$\hat{y} = 2x.$$

If

$$x = 1.01,$$

then

$$\hat{y} = 2.02.$$

The output changes much more smoothly.

Intuition. Large weights can make a model unstable and overly sensitive to small changes in the input. Regularization discourages unnecessarily large weights.

3 Regularized Loss Function

Without regularization, the model minimizes

$$\mathcal{L} = \mathcal{L}_{\text{data}}.$$

With regularization, the model minimizes

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \text{penalty on weights}.$$

This changes the training objective. The model no longer only asks:

How can I reduce prediction error?

It now asks:

How can I reduce prediction error while keeping the weights controlled?

4 L1 Regularization

L1 regularization adds the sum of absolute values of the weights to the loss:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_1 \sum_i |w_i|.$$

Here, $\lambda_1 \geq 0$ controls the strength of L1 regularization.

4.1 Effect of L1

L1 regularization can make some weights exactly zero. If a weight becomes zero, the corresponding feature has no effect on the prediction.

Suppose

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3.$$

If L1 regularization learns

$$w_3 = 0,$$

then

$$\hat{y} = w_1x_1 + w_2x_2 + 0x_3.$$

Since

$$0x_3 = 0,$$

the model becomes

$$\hat{y} = w_1x_1 + w_2x_2.$$

Therefore, x_3 is not physically deleted from the dataset, but it is removed from the model's decision-making.

Concrete example. Suppose

$$x_1 = 5, \quad x_2 = 80, \quad x_3 = 12345.$$

Without L1, suppose

$$\hat{y} = 5x_1 + 2x_2 + 0.4x_3.$$

Then

$$\hat{y} = 5(5) + 2(80) + 0.4(12345) = 25 + 160 + 4938 = 5123.$$

The student ID term dominates the prediction.

With L1, suppose the model learns

$$\hat{y} = 4.7x_1 + 1.6x_2 + 0x_3.$$

Then

$$\hat{y} = 4.7(5) + 1.6(80) + 0(12345) = 23.5 + 128 + 0 = 151.5.$$

Now x_3 has no effect.

5 L2 Regularization

L2 regularization adds the sum of squared weights to the loss:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_2 \sum_i w_i^2.$$

Here, $\lambda_2 \geq 0$ controls the strength of L2 regularization.

5.1 Effect of L2

L2 regularization strongly penalizes large weights because weights are squared:

$$2^2 = 4, \quad 10^2 = 100, \quad 100^2 = 10000.$$

Thus, L2 makes large weights expensive. It usually shrinks weights but does not make them exactly zero.

Model comparison. Suppose two models have similar data loss.

Model A:

$$\mathcal{L}_{\text{data}} = 1.0, \quad \mathbf{w} = [10, 8].$$

Model B:

$$\mathcal{L}_{\text{data}} = 1.2, \quad \mathbf{w} = [2, 1].$$

Without regularization, Model A is preferred because $1.0 < 1.2$.

With L2 regularization and $\lambda_2 = 0.01$:

$$\mathcal{L}_A = 1.0 + 0.01(10^2 + 8^2) = 1.0 + 1.64 = 2.64.$$

$$\mathcal{L}_B = 1.2 + 0.01(2^2 + 1^2) = 1.2 + 0.05 = 1.25.$$

Now Model B is preferred because it has smaller, more controlled weights.

6 What Happens During Backpropagation?

When regularization is added to the loss function, it affects the gradients used in backpropagation.

6.1 L2 Gradient

For a single weight,

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_2 w^2.$$

Then

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}_{\text{data}}}{\partial w} + 2\lambda_2 w.$$

Gradient descent updates the weight by

$$w_{\text{new}} = w - \eta \frac{\partial \mathcal{L}}{\partial w},$$

where η is the learning rate.

The extra term $2\lambda_2 w$ pulls w toward zero. This is why L2 is often called weight decay.

6.2 L1 Gradient

For a single weight,

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_1 |w|.$$

For $w \neq 0$,

$$\frac{d}{dw}|w| = \begin{cases} 1, & w > 0, \\ -1, & w < 0. \end{cases}$$

Therefore, L1 pushes positive weights downward and negative weights upward. In both cases, it pushes weights toward zero.

Key difference. L1 applies a roughly constant push toward zero, while L2 applies a push proportional to the current weight size.

7 Why L1 Creates Zeros but L2 Usually Does Not

The reason is found in the derivatives.

7.1 L1

For L1:

$$\lambda_1 |w|.$$

For $w > 0$, the derivative is

$$\lambda_1.$$

For $w < 0$, the derivative is

$$-\lambda_1.$$

This means L1 keeps pushing the weight toward zero with a constant force, even when the weight is already small.

7.2 L2

For L2:

$$\lambda_2 w^2.$$

The derivative is

$$2\lambda_2 w.$$

If w is large, the shrinkage force is large. If w is very small, the shrinkage force is tiny. Therefore, L2 shrinks weights smoothly but usually does not force them exactly to zero.

8 Using L1 and L2 Together: Elastic Net

When both L1 and L2 are added to the loss, the objective becomes

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_1 \sum_i |w_i| + \lambda_2 \sum_i w_i^2.$$

This is called Elastic Net regularization.

It combines two effects:

L1: creates sparsity by setting weak weights to zero,

L2: shrinks the remaining weights.

9 A One-Dimensional Toy Model

To prove the behavior mathematically, we study a one-weight objective:

$$J(w) = \frac{1}{2}(w - w_0^*)^2 + \lambda_1 |w| + \lambda_2 w^2.$$

Here:

- w is the weight being optimized.
- w_0^* is the best weight without regularization.
- w^* is the best weight after regularization.
- λ_1 is the L1 regularization strength.
- λ_2 is the L2 regularization strength.

The term

$$\frac{1}{2}(w - w_0^*)^2$$

means that the data wants w to stay close to the unregularized optimum w_0^* . The regularization terms pull w toward zero.

The factor $\frac{1}{2}$ is used only to simplify derivatives:

$$\frac{d}{dw} \left[\frac{1}{2}(w - w_0^*)^2 \right] = w - w_0^*.$$

10 Proof of the Elastic Net One-Dimensional Solution

We want to minimize

$$J(w) = \frac{1}{2}(w - w_0^*)^2 + \lambda_1|w| + \lambda_2w^2,$$

where

$$\lambda_1 \geq 0, \quad \lambda_2 \geq 0.$$

We will prove that the minimizer is

$$w^* = \frac{\text{sign}(w_0^*) \max(|w_0^*| - \lambda_1, 0)}{1 + 2\lambda_2}$$

10.1 Convexity

The function

$$\frac{1}{2}(w - w_0^*)^2$$

is convex, $|w|$ is convex, and w^2 is convex. Since a nonnegative weighted sum of convex functions is convex, $J(w)$ is convex. Therefore, any point satisfying the first-order optimality condition is a global minimizer.

10.2 Case 1: $w > 0$

If $w > 0$, then

$$|w| = w.$$

Thus

$$J(w) = \frac{1}{2}(w - w_0^*)^2 + \lambda_1w + \lambda_2w^2.$$

Differentiate:

$$\frac{dJ}{dw} = (w - w_0^*) + \lambda_1 + 2\lambda_2w.$$

Set the derivative to zero:

$$(w - w_0^*) + \lambda_1 + 2\lambda_2w = 0.$$

Rearrange:

$$\begin{aligned} w + 2\lambda_2 w - w_0^* + \lambda_1 &= 0, \\ (1 + 2\lambda_2)w &= w_0^* - \lambda_1. \end{aligned}$$

Therefore,

$$w = \frac{w_0^* - \lambda_1}{1 + 2\lambda_2}.$$

This case is valid only if $w > 0$. Since $1 + 2\lambda_2 > 0$, this requires

$$w_0^* - \lambda_1 > 0,$$

so

$$w_0^* > \lambda_1.$$

Thus, if $w_0^* > \lambda_1$,

$$w^* = \frac{w_0^* - \lambda_1}{1 + 2\lambda_2}$$

10.3 Case 2: $w < 0$

If $w < 0$, then

$$|w| = -w.$$

Thus

$$J(w) = \frac{1}{2}(w - w_0^*)^2 - \lambda_1 w + \lambda_2 w^2.$$

Differentiate:

$$\frac{dJ}{dw} = (w - w_0^*) - \lambda_1 + 2\lambda_2 w.$$

Set the derivative to zero:

$$(w - w_0^*) - \lambda_1 + 2\lambda_2 w = 0.$$

Rearrange:

$$\begin{aligned} w + 2\lambda_2 w - w_0^* - \lambda_1 &= 0, \\ (1 + 2\lambda_2)w &= w_0^* + \lambda_1. \end{aligned}$$

Therefore,

$$w = \frac{w_0^* + \lambda_1}{1 + 2\lambda_2}.$$

This case is valid only if $w < 0$. Since $1 + 2\lambda_2 > 0$, this requires

$$w_0^* + \lambda_1 < 0,$$

so

$$w_0^* < -\lambda_1.$$

Thus, if $w_0^* < -\lambda_1$,

$$w^* = \frac{w_0^* + \lambda_1}{1 + 2\lambda_2}$$

10.4 Case 3: $w = 0$

Not applicable for Exam

The function $|w|$ is not differentiable at $w = 0$, so we use the subgradient.

The subgradient of $|w|$ is

$$\partial|w| = \begin{cases} 1, & w > 0, \\ [-1, 1], & w = 0, \\ -1, & w < 0. \end{cases}$$

The subgradient of $J(w)$ is

$$\partial J(w) = (w - w_0^*) + \lambda_1 \partial|w| + 2\lambda_2 w.$$

At $w = 0$,

$$\partial J(0) = (0 - w_0^*) + \lambda_1 [-1, 1] + 2\lambda_2 (0).$$

Thus

$$\partial J(0) = -w_0^* + \lambda_1 [-1, 1].$$

For $w = 0$ to be optimal, zero must belong to this subgradient interval:

$$0 \in -w_0^* + \lambda_1 [-1, 1].$$

This is equivalent to

$$w_0^* \in \lambda_1 [-1, 1].$$

Therefore,

$$-\lambda_1 \leq w_0^* \leq \lambda_1,$$

or

$$|w_0^*| \leq \lambda_1.$$

Thus, if $|w_0^*| \leq \lambda_1$,

$$\boxed{w^* = 0.}$$

10.5 Piecewise Solution

Combining the three cases:

$$w^* = \begin{cases} \frac{w_0^* - \lambda_1}{1 + 2\lambda_2}, & w_0^* > \lambda_1, \\ 0, & |w_0^*| \leq \lambda_1, \\ \frac{w_0^* + \lambda_1}{1 + 2\lambda_2}, & w_0^* < -\lambda_1. \end{cases}$$

10.6 Compact Form

Now define

$$S_{\lambda_1}(w_0^*) = \text{sign}(w_0^*) \max(|w_0^*| - \lambda_1, 0).$$

This is called the soft-thresholding operator.

If $w_0^* > \lambda_1$, then

$$S_{\lambda_1}(w_0^*) = w_0^* - \lambda_1.$$

If $w_0^* < -\lambda_1$, then

$$S_{\lambda_1}(w_0^*) = w_0^* + \lambda_1.$$

If $|w_0^*| \leq \lambda_1$, then

$$S_{\lambda_1}(w_0^*) = 0.$$

Therefore, the piecewise solution can be written as

$$w^* = \frac{S_{\lambda_1}(w_0^*)}{1 + 2\lambda_2}$$

which gives

$$w^* = \frac{\text{sign}(w_0^*) \max(|w_0^*| - \lambda_1, 0)}{1 + 2\lambda_2}$$

This proves the equation.

11 Numerical Examples

11.1 Strong Weight

Let

$$w_0^* = 5, \quad \lambda_1 = 1, \quad \lambda_2 = 0.5.$$

Since

$$w_0^* > \lambda_1,$$

we use

$$w^* = \frac{w_0^* - \lambda_1}{1 + 2\lambda_2}.$$

Thus

$$w^* = \frac{5 - 1}{1 + 2(0.5)} = \frac{4}{2} = 2.$$

So the unregularized weight

$$w_0^* = 5$$

becomes

$$w^* = 2.$$

11.2 Weak Weight

Let

$$w_0^* = 0.6, \quad \lambda_1 = 1, \quad \lambda_2 = 0.5.$$

Since

$$|w_0^*| = 0.6 \leq 1,$$

we get

$$w^* = 0.$$

The weak weight is removed.

12 Final Takeaways

L1 regularization adds

$$\lambda_1 \sum_i |w_i|.$$

It can make weak weights exactly zero. Therefore, it can remove features from the model's decision-making.

L2 regularization adds

$$\lambda_2 \sum_i w_i^2.$$

It shrinks large weights and makes the model smoother, but usually does not make weights exactly zero.

L1 + L2 regularization gives Elastic Net:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_1 \sum_i |w_i| + \lambda_2 \sum_i w_i^2.$$

L1 performs thresholding; L2 performs shrinkage.

Method	Effect	Result
L1	Constant push toward zero; sharp corner at zero	Some weights become exactly zero
L2	Push proportional to weight size	Weights become smaller but usually nonzero
L1 + L2	Thresholding plus shrinkage	Weak weights become zero; remaining weights shrink